

COMPILER DESIGN LABORATORY · BCSE307P

CodeFlow

A Domain-Specific Language and Compiler for
Workflow Automation

Phase 2 Report

Core Compiler Construction — Semantic Analysis,
Symbol Table & Three-Address Code Generation

Author: Daksh Arora

Project type: Individual Project

Implementation language: TypeScript / Node.js

Date: September 25, 2026

1. Introduction & Scope

CodeFlow is a small, statically-typed, textual Domain-Specific Language (DSL) for expressing conditional workflow automation: declaring named `sensor` / `input` values, testing conditions over them, and triggering `action`, `alert`, or `log` effects in response. Phase 1 defined the language specification, grammar, and overall compiler architecture, and delivered an initial lexer/parser/AST prototype. This report covers **Phase 2** — **Core Compiler Construction**, which extends that prototype into a semantically validated compiler front end that emits an intermediate representation.

Per the project roadmap, Phase 2 targets the pipeline segment:

```
Source (.cflow) → Lexer → Tokens → Recursive-Descent Parser → AST →  
Semantic Analyzer → Symbol Table → Three-Address Code
```

Optimization and the execution engine remain out of scope for Phase 2 and are deferred to Phase 3. The compiler pipeline runs every implemented stage unconditionally — even when an earlier stage reports diagnostics — so that all intermediate artifacts (tokens, AST, symbol table, TAC) stay available for inspection in a single compilation pass, supporting the project's goal of making the source-to-execution transformation observable.

1.1 Summary of Phase 2 Deliverables

Module	Location	Status
Lexer	src/lexer/	Carried from Phase 1 unchanged
Parser	src/parser/parser.ts	Carried from Phase 1 unchanged
AST node types	src/ast/nodes.ts	Carried from Phase 1 unchanged
Symbol table	src/symbols/symbolTable.ts	new in Phase 2
Semantic analyzer	src/semantic/analyzer.ts	new in Phase 2
TAC instruction model	src/ir/instructions.ts	new in Phase 2
TAC generator	src/ir/generator.ts	new in Phase 2
TAC pretty-printer	src/ir/print.ts	new in Phase 2
Diagnostics model	src/diagnostics/diagnostics.ts	Extended with "semantic" stage
Pipeline wiring	src/compiler/pipeline.ts	Extended: analyzer + IR generator wired in
CLI	src/cli.ts	Extended: prints symbol tables + TAC
Web visualization	web/app.ts	Extended: symbol table + TAC panels
Optimizer	src/optimizer/	Phase 3
Execution engine	src/runtime/	Phase 3

2. Front End Recap: Lexer, Parser, AST

The lexer (`src/lexer/lexer.ts` , 245 lines) and recursive-descent parser (`src/parser/parser.ts` , 436 lines) were built in Phase 1 and are unchanged in Phase 2; they are included here because every later stage of Phase 2 operates directly on their output. The lexer converts raw source text into a `Token[]` , tracking line/column/offset for every token and reporting illegal characters, malformed numeric literals, and unterminated strings as lexical `CompilerErrors` . The parser is a hand-written recursive-descent implementation matching the operator precedence table from the Phase 1 grammar (`parseExpression` → `parseOr` → `parseAnd` → `parseNot` → `parseRelational` → `parseAdditive` → `parseMultiplicative` → `parseUnary` → `parsePrimary`), constructing the AST described below without performing any symbol resolution or type checking — that responsibility belongs entirely to the Phase 2 semantic analyzer, keeping syntax and semantic analysis cleanly separated as required by the project's architectural rules.

2.1 AST Node Model

The AST (`src/ast/nodes.ts`) is a discriminated union of plain TypeScript interfaces, each tagged by a `kind` field and carrying a `SourceLocation` ({ `line` , `column` }) so that every downstream diagnostic can point back to source text.

Node	Fields	Represents
Program	workflows: WorkflowDecl[]	The whole compilation unit
WorkflowDecl	name, declarations, statements	One workflow "... " { ... } block
Declaration	declKind (sensor input), name, type	A sensor / input declaration
WhenStatement	condition: Expr, body: Statement[]	when <expr> { ... }
OtherwiseStatement	body: Statement[]	otherwise { ... }
ActionStatement	name, args: Expr[]	action name(args)
AlertStatement	message: string	alert "... "
LogStatement	message: string	log "... "
LiteralExpr	literalType, value	Number / string / boolean literal
IdentifierExpr	name	A reference to a declared sensor/input
UnaryExpr	operator (- NOT), operand	Unary minus / logical NOT
BinaryExpr	operator, left, right	Arithmetic / relational / logical binary op

PrimitiveType = "number" | "string" | "boolean"; BinaryOperator = + - * / > < ≥ ≤ = ≠ AND OR; UnaryOperator = - | NOT.

3. Symbol Table

`SymbolTable` (`src/symbols/symbolTable.ts`, 55 lines) is new in Phase 2. It implements the symbol-table contract from the project specification — recording `name`, `kind`, `type`, and `scope` per declared symbol — with **one flat table per workflow**. This design follows directly from the language rule that a workflow is the only scope boundary in CodeFlow: declarations appear once, at the top of a workflow, and are never nested inside `when` / `otherwise` blocks, so a single `Map<string, SymbolEntry>` per workflow is sufficient to model scope resolution correctly without a scope-chain/stack structure.

3.1 Interface

```
class SymbolTable {
  readonly scope: string;           // the workflow name
  insert(declaration: SymbolDeclaration): boolean; // false if name already exists
  in this scope
  lookup(name: string): SymbolEntry | undefined; // scope-qualified resolution
  has(name: string): boolean;
  get symbols(): readonly SymbolEntry[];         // enumeration, for the symbol-
  table view
}
```

`insert` is the mechanism used to detect duplicate declarations: it refuses to overwrite an existing entry and returns `false`, which the semantic analyzer turns into a diagnostic rather than silently shadowing the first declaration. `sensor` and `input` declarations share one namespace per workflow (a workflow cannot declare a `sensor temperature` and an `input temperature` at the same time), matching real-world workflow-automation semantics where both are just named inputs to the condition logic.

3.2 Construction

The semantic analyzer constructs exactly one `SymbolTable` per `WorkflowDecl` while walking the program, inserting every declaration in that workflow's `declarations` list before analyzing its statements. All constructed tables are returned from `analyze()` as `SemanticResult.symbolTables`, giving the CLI and web visualization a per-workflow view (Symbol Table panel) with no extra bookkeeping.

4. Semantic Analyzer

`analyze(program)` (`src/semantic/analyzer.ts`, 217 lines) is the core new component of Phase 2. It performs a single recursive walk over the AST, enforcing every property the context-free grammar cannot express on its own: declaration checking, duplicate detection, scope checking, and a full static type-checking pass that assigns a `PrimitiveType` to every expression node.

4.1 Declaration & Scope Checking

Every `IdentifierExpr` is resolved against the enclosing workflow's `SymbolTable`. An identifier with no matching declaration is reported as `Undefined identifier '<name>'` at its source location. Because `CodeFlow` has exactly one scope level (the workflow), there is no scope-chain walk — resolution is a single `lookup()` call against the current workflow's table, and a declaration made in one workflow is never visible from another (verified by the *"does not leak a declaration from one workflow into another"* test).

4.2 Duplicate Declaration Checking

While populating each workflow's symbol table, a second declaration reusing an already-declared name is reported as `Duplicate declaration of '<name>' in workflow "<workflow>"`, and the original entry is left untouched in the table (so subsequent lookups still resolve against the first, valid declaration).

4.3 Type Checking

The analyzer performs bottom-up type inference over every `Expr`, returning a `PrimitiveType` | `undefined` (`undefined` marks an expression that already failed to type-check, which stops errors from cascading further up the same expression tree). Successfully typed expressions are recorded in `SemanticResult.types: Map<Expr, PrimitiveType>` for potential reuse by later stages.

Operator class	Operators	Required operand type(s)	Result type
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code>	<code>number</code> , <code>number</code>	<code>number</code>
Relational	<code>></code> <code><</code> <code>≥</code> <code>≤</code>	<code>number</code> , <code>number</code>	<code>boolean</code>
Equality	<code>=</code> <code>≠</code>	both operands the <i>same</i> primitive type	<code>boolean</code>
Logical (binary)	<code>AND</code> <code>OR</code>	<code>boolean</code> , <code>boolean</code>	<code>boolean</code>
Logical (unary)	<code>NOT</code>	<code>boolean</code>	<code>boolean</code>
Arithmetic (unary)	<code>-</code>	<code>number</code>	<code>number</code>

Additionally, the condition expression of every `when` statement is required to type as `boolean` (The condition of a 'when' statement must be of type 'boolean', but found '<type>'); a non-boolean condition such as `when temperature { ... }` is rejected even though `temperature` itself is a validly-declared identifier.

Accepted programs (excerpt from the test suite)

```
number > number      // relational, numeric operands
number + number      // arithmetic
boolean AND boolean  // logical
string = string       // equality, matching types
boolean = boolean     // equality, matching types
```

Rejected programs (excerpt from the test suite)

```
"hello" > 10          // relational requires number, number
true + 5              // arithmetic requires number, number
10 AND 5              // logical requires boolean, boolean
temperature AND 10    // temperature: number, AND requires boolean
```

4.4 Error Recovery

Analysis does not stop at the first diagnostic. Mirroring the parser's own recovery strategy, the analyzer keeps walking the remaining declarations, statements, and workflows after reporting an error, so a single compilation pass can surface every semantic problem in a program rather than forcing the user to fix and recompile one error at a time. Errors are still prevented from *cascading* within one expression: once an operand fails to resolve or type-check, the enclosing expression returns `undefined` instead of guessing a type and reporting a second, spurious error on top of the first.

4.5 Diagnostic Output

Every semantic diagnostic is emitted as a `CompilerError` with `stage: "semantic"` and the offending node's `line / column`, consistent with the shared cross-cutting error model used by the lexer and parser:

```
type CompilerError = {
  stage: "lexical" | "syntax" | "semantic" | "runtime";
  message: string;
  line?: number;
  column?: number;
  length?: number;
};
```

5. Three-Address Code Generation

5.1 Instruction Model

`src/ir/instructions.ts` defines TAC as a discriminated union (`IRInstruction`) of nine structured instruction shapes, rather than free-form strings, so the instruction stream stays type-safe and straightforward for the Phase 3 optimizer and execution engine to pattern match over:

```
type IRInstruction =
  | AssignInstruction           // dest = value
  | BinaryInstruction          // dest = left OP right
  | UnaryInstruction           // dest = OP operand
  | ConditionalJumpInstruction // IF_FALSE cond GOTO label
  | JumpInstruction            // GOTO label
  | LabelInstruction           // LABEL label
  | CallInstruction            // CALL name(args)
  | AlertInstruction           // ALERT "message"
  | LogInstruction;            // LOG "message"
```

Operands are likewise a closed union:

```
type Operand =
  | { kind: "Temp"; id: number } // compiler-generated temporary
  | { kind: "Var"; name: string } // a declared sensor/input
  | { kind: "Const"; value: number | string | boolean }; // a literal value
```

The generator always materializes `Var` and `Const` operands into a `Temp` via an `Assign` instruction before they are used in any computation, so every instruction only ever references the result of at most one prior operation — true three-address discipline, with no instruction doing more than "one operator's worth" of work. Each workflow lowers to its own `WorkflowIR = { workflowName, instructions: IRInstruction[] }`.

5.2 Generation Algorithm

`generateIR(program)` (`src/ir/generator.ts`, 152 lines) lowers each workflow's AST independently — workflows never call one another in `CodeFlow`, so per-workflow temporary (`t1`, `t2`, ...) and label (`L1`, `L2`, ...) counters simply reset at the start of each workflow.

- **Expressions** always lower to a `Temp` operand: a `LiteralExpr` or `IdentifierExpr` is first assigned into a fresh temporary, then combined through `Binary` / `Unary` instructions whose operands are themselves temporaries.
- **when alone** lowers to: evaluate the condition, `IfFalse cond GOTO endLabel`, the body, then `Label endLabel`.

- **when immediately followed by otherwise** in the same statement list lowers to a structured if/else: `IfFalse cond GOTO elseLabel`, the `when` body, `Goto endLabel`, `Label elseLabel`, the `otherwise` body, `Label endLabel`. This pairing is purely structural — adjacency of the two statements in the list — and introduces no new grammar rule, matching the language's definition of `otherwise` as a fallback executed only when the immediately preceding `when` 's condition is false.
- **An orphaned otherwise** (no directly preceding `when`) has no condition to fall back on, so its body is lowered unconditionally.
- **action / alert / log** lower directly to `Call / Alert / Log` instructions; action arguments are lowered like any other expression and passed as a list of operands.

5.3 Worked Example

For the canonical example program `examples/cooling.cflow`:

```
workflow "CoolingSystem" {
  sensor temperature : number

  when temperature > 35 {
    alert "High temperature"
    action start_fan()
  }

  otherwise {
    log "Temperature normal"
  }
}
```

the generator emits (as rendered by the CLI / `formatWorkflowIR`):

```
workflow "CoolingSystem":
  t1 = temperature
  t2 = 35
  t3 = t1 > t2
  IF_FALSE t3 GOTO L1
  ALERT "High temperature"
  CALL start_fan()
  GOTO L2
LABEL L1
  LOG "Temperature normal"
LABEL L2
```

`src/ir/print.ts` (43 lines) is a pure formatter that renders this human-readable text from a `WorkflowIR`; it is consulted only by the CLI and the web visualization and is never read back by any compiler stage, preserving the rule that IR consumers act on the structured `IRInstruction[]`, not on its printed form.

6. Pipeline, CLI & Visualization Integration

6.1 Compiler Pipeline

`compile(source: string): CompileResult` (`src/compiler/pipeline.ts`) wires the four implemented stages together:

```
Lexer.tokenize()
  → Parser.parse()
  → analyze(program)      // semantic analyzer + symbol tables
  → generateIR(program)    // TAC generator

return { tokens, program, symbolTables, ir, errors }
```

Every stage always runs over whatever artifact the previous stage produced, even when earlier stages already reported errors (for example, TAC is still generated from a program that failed semantic analysis). This keeps every intermediate artifact inspectable in one pass, which is what the visualization layer relies on. All lexical, syntax, and semantic diagnostics are concatenated into a single `CompilerError[]` in stage order.

6.2 CLI

`src/cli.ts` compiles a given `.cflow` file and prints, in order: the token stream, the AST as formatted JSON, one symbol table per workflow, the generated TAC for every workflow, and finally any lexical/syntax/semantic diagnostics (or a confirmation that none were found).

6.3 Web Visualization

The browser UI (`web/app.ts`) gained a Symbol Table panel and a Three-Address Code panel in Phase 2, alongside the Phase 1 Tokens and AST panels. All panels re-render from a single `compile()` call whenever the source text changes, so the tool continues to demonstrate the full pipeline observably rather than only the final result.

7. Testing & Verification

Testing mirrors the compiler stages. Phase 2 added three new suites (symbol table, semantic analyzer, IR generator) and extended the pipeline suite to exercise the new stages end to end. The full suite currently comprises **7 test files and 81 passing tests** (`npm test` , via Vitest).

7.1 Symbol Table — `tests/symbols/symbolTable.test.ts`

- inserts and looks up a symbol
- returns `undefined` for an unknown identifier
- rejects a duplicate declaration and leaves the original entry intact
- treats sensor and input declarations as sharing one namespace
- exposes all inserted symbols
- keeps separate scopes independent

7.2 Semantic Analyzer — `tests/semantic/analyzer.test.ts`

- accepts valid sensor and input declarations with no errors
- reports a duplicate declaration in the same workflow
- reports an undefined identifier referenced in a condition
- reports an undefined identifier used as an action argument
- does not report undeclared action names (actions are not declarations)
- does not leak a declaration from one workflow into another
- resolves a declaration inside nested `when/otherwise` blocks of the same workflow
- accepts `number > number` , `number + number` , `boolean AND boolean` , `string = string` , `boolean = boolean`
- rejects `"hello" > 10` , `true + 5` , `10 AND 5` , `temperature AND 10`
- accepts equality between two operands of the same type / rejects equality between operands of different types
- rejects a non-boolean `when` condition
- does not cascade a second error when the condition already failed to resolve
- accepts unary minus on a number and `NOT` on a boolean / rejects unary minus on a non-number

7.3 TAC Generator — `tests/ir/generator.test.ts`

- lowers arithmetic expressions into named temporaries
- lowers a comparison into a Binary instruction over materialized operands
- emits a matching conditional jump and label for a `when` without `otherwise`
- lowers a `when / otherwise` pair into an if/else with a shared end label
- does not fall through into the otherwise branch when the when branch executes
- treats an `otherwise` with no preceding `when` as unconditional
- generates independent temp/label numbering for multiple `when` statements

- lowers action calls with multiple materialized arguments / with no arguments
- lowers alert and log statements directly, without temporaries
- lowers a unary `NOT` and unary minus
- lowers nested when/otherwise blocks
- generates one instruction sequence per workflow, independently numbered
- formats a full workflow as readable text

7.4 Pipeline (end-to-end) — `tests/compiler/pipeline.test.ts`

- compiles the canonical cooling-system example with no errors
- compiles the canonical order-processing example with no errors
- reports a syntax error for the missing-brace example
- reports a lexical error for the invalid-token example
- reports a semantic error for the undefined-identifier example
- reports a semantic error for the invalid-type example
- populates a symbol table per workflow
- generates TAC for the canonical cooling-system example

Lexer (`tests/lexer/lexer.test.ts`), parser (`tests/parser/parser.test.ts`), and web UI (`tests/web/app.test.ts`) suites are carried from Phase 1 and continue to pass unmodified, confirming Phase 2 did not regress existing behavior — a project requirement whenever a new stage is added on top of an existing one.

8. Required Error Test Cases

The malformed CodeFlow programs required by the project specification are maintained under `examples/errors/` and exercised by the pipeline test suite, each producing a stage-specific, source-located `CompilerError` instead of a generic failure:

File	Defect	Diagnostic stage
<code>undefined-identifier.cflow</code>	Condition references a name never declared in the workflow	semantic
<code>invalid-type.cflow</code>	<code>boolean + number</code> in a <code>when</code> condition	semantic
<code>missing-brace.cflow</code>	Unclosed <code>when</code> block	syntax
<code>invalid-token.cflow</code>	Illegal character (<code>@</code>) in source	lexical

9. Limitations & Phase 3 Roadmap

Phase 2 deliberately does not implement optimization or execution; TAC generated today is not yet consumed by anything, and no part of the CLI, web UI, or documentation claims otherwise. Phase 3 will add, directly on top of the artifacts described in this report:

1. **Optimizer** (`src/optimizer/`) — constant folding (e.g. `10 + 20 → 30`), dead-code elimination, and unreachable-code elimination, each tested as input-IR $\rightarrow$ optimized-IR with execution semantics verified to be unchanged.
2. **Execution engine** (`src/runtime/`) — an interpreter over optimized TAC that maintains runtime values, evaluates expressions, follows labels/branches, dispatches actions, emits alerts/logs, and produces an execution trace, driven by sensor/input values supplied at run time (CodeFlow has no real physical sensors).
3. **Visualization** — a before/after IR comparison view and an execution trace panel, completing the artifact list (Source, Tokens, AST, Symbol Table, Semantic Diagnostics, Generated TAC, Optimized TAC, Execution Trace).
4. **Comprehensive regression testing** across the full pipeline and final project documentation.

10. Conclusion

Phase 2 turns the Phase 1 syntax prototype into a semantically validated compiler front end: a scoped symbol table, a type-checking semantic analyzer that enforces every rule from the language specification, and a structured Three-Address Code generator that lowers validated CodeFlow programs into an inspectable, executable-shaped intermediate representation. All new modules are independently unit-tested (33 new tests across the symbol table, semantic analyzer, and IR generator suites), the existing lexer/parser/web suites continue to pass unmodified, and the full pipeline — source through TAC — is exercised end to end, including the required malformed-program cases. This leaves Phase 3 free to focus purely on optimization and execution, without needing to revisit the front end.